

Deep Learning Techniques for Autonomous Spacecraft Guidance During Proximity Operations

Lorenzo Federici,^{*} Boris Benedikter,[†] and Alessandro Zavoli[‡]

Sapienza University of Rome, 00184 Rome, Italy

<https://doi.org/10.2514/1.A35076>

This paper investigates the use of deep learning techniques for real-time optimal spacecraft guidance during terminal rendezvous maneuvers, in presence of both operational constraints and stochastic effects, such as an inaccurate knowledge of the initial spacecraft state and the presence of random in-flight disturbances. The performance of two well-studied deep learning methods, behavioral cloning (BC) and reinforcement learning (RL), is investigated on a linear multi-impulsive rendezvous mission. To this aim, a multilayer perceptron network, with custom architecture, is designed to map any observation of the actual spacecraft relative position and velocity to the propellant-optimal control action, which corresponds to a bounded-magnitude impulsive velocity variation. In the BC approach, the deep neural network is trained by supervised learning on a set of optimal trajectories, generated by routinely solving the deterministic optimal control problem via convex optimization, starting from scattered initial conditions. Conversely, in the RL approach, a state-of-the-art actor–critic algorithm, proximal policy optimization, is used for training the network through repeated interactions with the stochastic environment. Eventually, the robustness and propellant efficiency of the obtained closed-loop control policies are assessed and compared by means of a Monte Carlo analysis, carried out by considering different test cases with increasing levels of perturbations.

Nomenclature

a	=	action
a_p	=	random acceleration
B	=	actuation matrix
$\mathbb{E}[a]$	=	expected value of a random variable a with respect to τ
J	=	merit index
M	=	number of time steps per training episode
o	=	observation
R	=	reward
r	=	position vector
S^E	=	eyelash set
S^F	=	funnel set
T	=	total number of training steps
t	=	time
$\mathcal{U}(a, b)$	=	uniform distribution in interval $[a, b]$
V^π	=	value function
v	=	velocity vector
x	=	state
β	=	half-angle of the visibility cone
γ	=	discount factor
ΔV	=	total velocity variation
Δv	=	impulsive velocity variation
δx	=	random state perturbation
ε	=	terminal constraints satisfaction tolerance
θ	=	neural network parameters
π	=	control policy
τ	=	trajectory
Φ	=	state transition matrix

ω = angular velocity of the target

Subscript

k = value at k th time step

Superscripts

T = transpose

$\star$ = optimal value

$\sim$ = reference value

I. Introduction

AUTONOMOUS orbital rendezvous between two spacecraft is a key technology for many emerging mission concepts, such as active debris removal, sample return, and on-orbit servicing, to name a few. At present, close proximity operations generally require intensive cooperation between both vehicles, complex navigation systems, or a man-in-the-loop to ensure the success of the maneuver [1]. Conversely, future concepts are mainly based on small spacecraft that usually have to reach for a passive target, i.e., with minimal or none maneuvering capability, and that cannot rely on elaborate instrumentation and human involvement. To meet this new demand, the definition of computationally efficient and robust guidance algorithms has attracted the attention of many researchers in the aerospace community.

Because open-loop orbital maneuvers may show high sensitivity to unpredicted operating conditions, autonomous guidance must be embedded in a closed-loop architecture that accounts for the real-time measurements provided by the navigation system. Guidance algorithms can be classified according to the rationale used to generate the nominal trajectory for the inner control loops. For instance, a typical criterion is the maximization of a performance index, which requires the formulation and solution of an optimal control problem (OCP). Because of the limited on-board computational power, traditional optimal guidance algorithms are based exclusively on analytical solutions. In this manner, only a limited number of algebraic operations are needed for evaluating closed-form guidance laws. Despite the continuous effort to extend the applicability of the analytical approach to a growing number of scenarios, closed-form solutions can be derived only for a limited class of problems and are inherently specific to the prescribed state representation, dynamic model, and cost function. So, along with the latest progress in

Received 26 January 2021; revision received 14 March 2021; accepted for publication 28 April 2021; published online 30 July 2021. Copyright © 2021 by Lorenzo Federici, Boris Benedikter, and Alessandro Zavoli. Published by the American Institute of Aeronautics and Astronautics, Inc., with permission. All requests for copying and permission to reprint should be submitted to CCC at www.copyright.com; employ the eISSN 1533-6794 to initiate your request. See also AIAA Rights and Permissions www.aiaa.org/randp.

^{*}Ph.D. Student, Department of Mechanical and Aerospace Engineering, Via Eudossiana 18; lorenzo.federici@uniroma1.it.

[†]Ph.D. Student, Department of Mechanical and Aerospace Engineering, Via Eudossiana 18; boris.benedikter@uniroma1.it.

[‡]Researcher, Department of Mechanical and Aerospace Engineering, Via Eudossiana 18; alessandro.zavoli@uniroma1.it.

computing hardware, computational guidance gained increasing interest in the aerospace community [2]. Differently from the traditional algorithms, this branch intensively relies on on-board computation for trajectory generation. A guidance framework that falls within the computational guidance is model predictive control (MPC), which consists in recursively solving an OCP updated at each guidance step with the available state measurements. The PRISMA technology demonstration mission, launched in 2010, successfully tested in-orbit the capabilities of model predictive controllers for rendezvous and formation flying [3]. However, except for problem instances that can be naturally solved via linear, quadratic, or convex programming, carefully tailored sequential optimization methods must be implemented to make problems characterized by general nonconvex constraints computationally tractable. This can significantly slow down the optimization process and limit the closed-loop update frequency, thus hindering the algorithm robustness.

In the past few years, deep learning techniques proved to be successful when applied to solve a wide variety of control problems across several research areas, such as robotics [4] and automotive [5], but also in the aerospace field, in particular for what concerns the path planning of autonomous unmanned aerial systems (UAS) [6,7] and agile flight guidance [8]. More recently, the power of deep learning has become clear also to those involved in space missions. In this context, the use of a so-called Guidance and Control Network, or G&CNet, was proposed by Izzo et al. [9] as an innovative way to provide the spacecraft with real-time guidance and control capabilities, by exploiting the low computational times and the high accuracy in function approximation of a deep neural network (DNN). To this end, the DNN is trained to imitate, or clone, as best as possible an “expert behavior,” provided in terms of optimal state–action pairs that come from the numerical solution of a suitably defined OCP. During the training, the DNN learns a map from any spacecraft (observed) state (e.g., position and velocity) to the corresponding optimal control (e.g., the thrust magnitude and direction), by leveraging the demonstrations provided by the OCP solver. This approach, formally known as behavioral cloning (BC), essentially reduces the problem to a supervised learning task, which can be tackled effectively using state-of-the-art algorithms and well-established open-source libraries [10], provided that a sufficiently large data set of expert (i.e., optimal) trajectories for the problem at hand is available. BC has been recently applied to a variety of spaceflight mechanics problems. Typical applications include the approximation of optimal state-feedback control laws for interplanetary transfers [11] and planetary soft-landing maneuvers [12,13], as well as the real-time onboard generation of a high number of optimal trajectories for either asteroid landing [14,15] or atmospheric reentry of hypersonic vehicles [16,17]. BC-based controllers, trained on a limited number of optimal solutions provided by an indirect approach, were also used for the computation of minimum-time trajectories of a solar sail spacecraft [18] and mass-optimal trajectories of a low-thrust spacecraft [19]. Furthermore, in conjunction with a data set aggregation approach (DAGGER) [20], BC has been effectively exploited to train a convolutional neural network to successfully control a lander during a powered descent on the lunar surface, using just raw camera images as observations [21].

Reinforcement learning (RL) represents an alternative way to teach a DNN to optimally solve a control task. In RL, training data are collected online by the network itself (the “agent”), through direct interactions with the external world (the “environment”), which, in space guidance applications, consist of simulations of the considered mission scenario. The only external feedback the agent receives, at training time, is a numerical reward, which helps it understanding how good or how bad it is currently performing, and, consequently, how to improve its behavior. Thus, the goal of RL is to learn the control policy that maximizes the (expected) cumulative reward along a trajectory. This approach differs significantly from BC, where the network learns by looking at a precomputed (labeled) set of data. RL has the clear advantage over BC of not requiring the a priori computation of any optimal trajectory for populating the training data set. As a drawback, the RL process is considerably slower, as the collected data are correlated to each other. Furthermore, this

correlation can lead to vicious circles: if the agent starts collecting poor quality data (e.g., trajectories with no rewards), it has little to no chance of knowing how to improve its performance, and it will continue to accumulate bad trajectories. Recent applications of RL in spaceflight mechanics problems concern the on-board guidance of a spacecraft during low-thrust interplanetary [22] and LEO–GEO [23] transfers, 6-DoF planetary landing guidance with application to a powered descent on Mars surface [24], autonomous guidance during transfers in the cislunar environment [25–27], and the design of guidance algorithms for interception [28] or rendezvous [29,30] maneuvers. In almost all of the aforementioned cases, the adopted algorithm for training the agent is proximal policy optimization (PPO) [31], being considered almost unanimously the state-of-the-art for the solution of continuous-control problems, somehow superseding Deep Deterministic Policy Gradient (DDPG) [32], which was used in earlier works on robotics.

Furthermore, the combined power of recurrent neural networks (RNNs) and RL brought to the definition of what is commonly known as reinforcement meta-learning, or “learning to learn.” Thanks to the unique ability of RNNs to retain information about the temporal variation of the observations collected over the training in their internal or hidden states, meta-learning methods have achieved improved performance in environments characterized by unknown [33], uncertain [34], and/or partially observable [35] dynamics.

This paper proposes to investigate and compare the use of BC and RL to provide a chaser spacecraft with autonomous guidance capabilities during a terminal rendezvous maneuver with a non-cooperative target. The approach maneuver starts when the chaser is sufficiently close to the target and terminates when it reaches a terminal position that is located slightly away from the target. The chaser trajectory is forced inside a visibility cone with vertex in the target position, thus incorporating a traditional, yet realistic, operational constraint. Furthermore, the presence of uncertain initial conditions and that of a perturbing random acceleration are also accounted for. The (deterministic) rendezvous problem is first formulated as an OCP. A second-order cone programming formulation is devised, which allows for the use of highly efficient interior-point algorithms that feature polynomial-time complexity and guarantee convergence toward the global optimum regardless of the initialization and of the specific problem instance. A sufficiently large set of expert demonstrations is then created by repeatedly solving the convex rendezvous problem with scattered initial conditions and used to train a DNN by BC. On the other hand, PPO algorithm is used for directly training the DNN in the stochastic environment, according to the RL paradigm. Eventually, the performance and robustness of the two networks are assessed and compared by means of a Monte Carlo analysis, performed in presence of the considered stochastic effects.

II. Problem Statement

In this section, the rendezvous problem is described and formulated as an OCP. First, a deterministic model is employed; then, the probability models used to account for uncertainties on the initial conditions and on the system dynamics are introduced.

A. Deterministic Scenario

Let us consider an active spacecraft, named the *chaser*, that has to come in close proximity of a passive spacecraft, named the *target*. The target is assumed to fly on a Keplerian circular orbit with angular velocity ω . The chaser state $\mathbf{x}$ at any time is completely described by its relative position $\mathbf{r} = [x \ y \ z]^T$ and relative velocity $\mathbf{v} = [\dot{x} \ \dot{y} \ \dot{z}]^T$ with respect to the target, i.e., $\mathbf{x} = [\mathbf{r}^T \ \mathbf{v}^T]^T$. Position and velocity are conveniently expressed in a radial-transverse-normal (RTN) reference frame centered in the target spacecraft, with the x axis (R-bar) in outgoing radial direction, the y axis (V-bar) in the direction of the velocity vector of the target, and the z axis (H-bar) in the direction of the angular momentum of the target, as shown in Fig. 1.

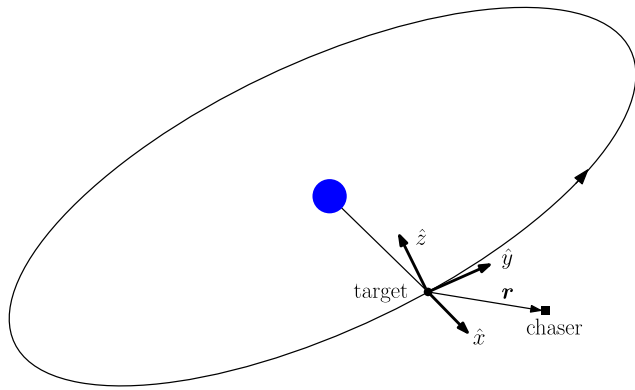


Fig. 1 RTN reference frame.

The linearized relative motion of the chaser with respect to the target can be described by using the Hill–Clohessy–Wiltshire (HCW) equations [36]. Being linear differential equations, their solution can be written in terms of the state transition matrix (STM) $\Phi(\tau)$ that links the spacecraft state $\mathbf{x}(t)$ at any time t with the spacecraft state $\mathbf{x}(t_0)$ at a previous time $t_0 = t - \tau$:

$$\mathbf{x}(t) = \Phi(\tau)\mathbf{x}(t_0) \quad (1)$$

Equation (1) can be decomposed in position and velocity components as

$$\begin{bmatrix} \mathbf{r}(t) \\ \mathbf{v}(t) \end{bmatrix} = \begin{bmatrix} M(\tau) & N(\tau) \\ S(\tau) & T(\tau) \end{bmatrix} \begin{bmatrix} \mathbf{r}(t_0) \\ \mathbf{v}(t_0) \end{bmatrix} \quad (2)$$

where $M(\tau)$, $N(\tau)$, $S(\tau)$, and $T(\tau)$ are 3×3 matrices that compose the STM, here reported for the sake of completeness

$$\begin{aligned} M &= \begin{bmatrix} 4 - 3 \cos \omega\tau & 0 & 0 \\ 6(\sin \omega\tau - \omega\tau) & 1 & 0 \\ 0 & 0 & \cos \omega\tau \end{bmatrix} \\ N &= \begin{bmatrix} \frac{1}{\omega} \sin \omega\tau & \frac{2}{\omega} (1 - \cos \omega\tau) & 0 \\ -\frac{2}{\omega} (1 - \cos \omega\tau) & \frac{4}{\omega} \sin \omega\tau - 3\tau & 0 \\ 0 & 0 & \frac{1}{\omega} \sin \omega\tau \end{bmatrix} \\ S &= \begin{bmatrix} 3\omega \sin \omega\tau & 0 & 0 \\ -6\omega \sin \omega\tau & 0 & 0 \\ 0 & 0 & -\omega \sin \omega\tau \end{bmatrix} \\ T &= \begin{bmatrix} \cos \omega\tau & 2 \sin \omega\tau & 0 \\ -2 \sin \omega\tau & 4 \cos \omega\tau - 3 & 0 \\ 0 & 0 & \cos \omega\tau \end{bmatrix} \end{aligned} \quad (3)$$

A bounded-magnitude impulsive thrust model is considered. The trajectory is discretized over a grid of $M + 1$ points, uniformly spaced in time between the initial time $t_0 = 0$ and the final time t_f

$$0 = t_0 < t_1 < \dots < t_M = t_f \quad (4)$$

so that the grid time step is $\Delta t = t_{k+1} - t_k = t_f/M$, $k = 0, \dots, M - 1$. Subscript “ k ” is used to identify quantities at time t_k , whereas superscripts “ $-$ ” and “ $+$ ” are used to distinguish the spacecraft state immediately before and after the burns. An impulsive maneuver $\Delta \mathbf{v}_k = [\Delta v_{x,k} \ \Delta v_{y,k} \ \Delta v_{z,k}]^T$ is performed by the chaser spacecraft at any node of the time grid. The magnitude of any impulse is bounded by a maximum value $\Delta v_{\max}$, related to the maximum thrust available

$$\|\Delta \mathbf{v}_k\| \leq \Delta v_{\max} \quad (5)$$

Because the chaser spacecraft moves under linearized dynamics between any two impulses, its state can be propagated analytically between any two successive times of the grid with a time-discrete closed-form transition function f :

$$\mathbf{x}_{k+1}^- = f(\mathbf{x}_k^-, \Delta \mathbf{v}_k) = \Phi(\Delta t)\mathbf{x}_k^+ \quad (6)$$

for $k = 0, \dots, M - 1$, where

$$\mathbf{x}_k^+ = \mathbf{x}_k^- + B\Delta \mathbf{v}_k \quad (7)$$

with $B = [\mathbf{0}_{3 \times 3} \ \mathbf{I}_{3 \times 3}]^T$ the actuation matrix. $\mathbf{0}_{m \times n}$ and $\mathbf{I}_{m \times n}$ denote the null and identity matrix of size $m \times n$.

During the whole maneuver, the direction of approach of the chaser is limited by the presence of a visibility cone constraint, which guarantees that the chaser spacecraft is always visible from the target. A three-dimensional visibility cone with half-angle $\tilde{\beta}$, as the one shown in Fig. 2, is considered in this work. The cone originates from the target, is aligned along $\mathbf{V}$ -bar, and widens toward negative y values. The chaser motion can be forced inside the feasible region at any time t_k by using the following set of four linear inequality constraints [37]

$$C\mathbf{x}_k^- \leq \mathbf{0}_{4 \times 1} \quad (8)$$

where

$$C = \begin{bmatrix} 1 & \tan \tilde{\beta} & 0 & 0 & 0 & 0 \\ -1 & \tan \tilde{\beta} & 0 & 0 & 0 & 0 \\ 0 & \tan \tilde{\beta} & 1 & 0 & 0 & 0 \\ 0 & \tan \tilde{\beta} & -1 & 0 & 0 & 0 \end{bmatrix} \quad (9)$$

The chaser trajectory is supposed to start from given initial conditions $\mathbf{x}_i = [\mathbf{r}_i^T \ \mathbf{v}_i^T]^T$ and terminate at assigned final conditions $\mathbf{x}_f = [\mathbf{r}_f^T \ \mathbf{v}_f^T]^T$. So, the following boundary conditions must hold

$$\mathbf{x}_0^- = \mathbf{x}_i \quad (10)$$

$$\mathbf{x}_M^+ = \mathbf{x}_f \quad (11)$$

The objective of the optimization procedure is to minimize the total velocity variation performed with the on-board thrusters

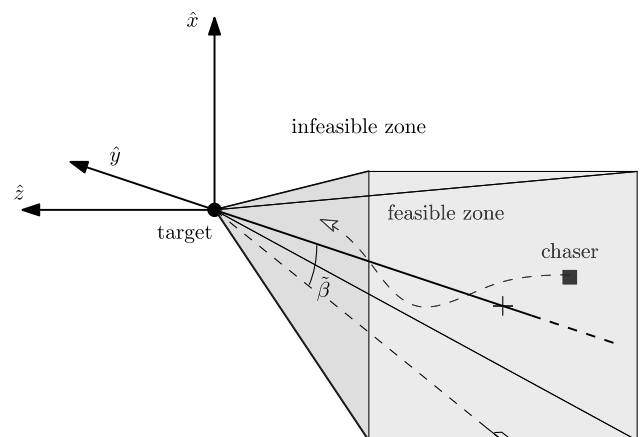


Fig. 2 Visibility cone constraint.

$$\Delta V = \sum_{k=0}^M \|\Delta \mathbf{v}_k\| \quad (12)$$

As a result, the deterministic OCP is

$$\begin{aligned} \min_{\Delta \mathbf{v}_k} \quad & \Delta V \\ \text{s.t.} \quad & (6) - (9), (11) - (12) \end{aligned} \quad (13)$$

Notice that the OCP of Eq. (13) is a quadratically constrained quadratic programming problem that can be equivalently formulated as a second-order cone programming (SOCP) problem. More precisely, by introducing a nonnegative auxiliary variable $\sigma_k \in [0, \Delta v_{\max}]$, the quadratic constraints in Eq. (5) and the objective function in Eq. (12) can be posed as $\|\Delta \mathbf{v}_k\| \leq \sigma_k$ for $k = 0, \dots, M$, and $\Delta V = \sum_{k=0}^M \sigma_k$, respectively [38]. The SOCP formulation is particularly interesting as it allows the use of state-of-the-art solvers for the numerical solution of the problem, such as GUROBI [39].

B. Stochastic Effects

In practice, the initial chaser position may be slightly different from the nominal one. For this reason, we assume that the actual initial condition $\mathbf{x}_0^-$ can be randomly scattered about the nominal condition $\mathbf{x}_i$, i.e.,

$$\mathbf{x}_0^- = \mathbf{x}_i + \delta \mathbf{x}_i \quad (14)$$

$\delta \mathbf{x}_i$ is a random perturbation sampled from a uniform distribution

$$\delta \mathbf{x}_i \sim \mathcal{U}(-\delta \mathbf{x}_i^{\max}, \delta \mathbf{x}_i^{\max}) \quad (15)$$

where $\delta \mathbf{x}_i^{\max} = [(\delta \mathbf{r}_i^{\max})^T \ (\delta \mathbf{v}_i^{\max})^T]^T$, and $\mathcal{U}(a, b)$ denotes a uniform distribution of lower and upper bound a and b , respectively.

Also, the actual dynamics may differ from the nominal linear dynamics in Eq. (6), due to the presence of unmodeled accelerations, badly estimated model parameters, and control actuation errors. As proposed by Richards and How [40], we model the overall effect of these uncertainty sources by introducing a random velocity perturbation $\Delta \mathbf{v}_p = \mathbf{a}_p \Delta t$ applied at the beginning of each time step as

$$\mathbf{x}_k^+ = \mathbf{x}_k^- + B(\Delta \mathbf{v}_k + \Delta \mathbf{v}_p) \quad (16)$$

where $\mathbf{a}_p$ is a random acceleration sampled at each time step from a uniform distribution

$$\mathbf{a}_p \sim \mathcal{U}(-\mathbf{a}_p^{\max}, \mathbf{a}_p^{\max}) \quad (17)$$

III. Deep Learning for Spacecraft Guidance

A closed-loop guidance can be used to drive the system toward the desired final condition in spite of the stochastic effects of Eqs. (14) and (16). Let us introduce a vector of observations $\mathbf{o}_k \in \mathcal{O}$ (i.e., measurements), which represents the knowledge of the system state available to the controller at time t_k , and an output-feedback control policy $\pi: \mathcal{O} \rightarrow \mathcal{A}$, which maps every observation $\mathbf{o}_k$ to the corresponding control (or action) $\mathbf{a}_k \in \mathcal{A}$, i.e.,

$$\mathbf{a}_k = \pi(\mathbf{o}_k) \quad (18)$$

In the present work the controller is assumed to have perfect knowledge of the system state at any time, hence

$$\mathbf{o}_k = [(\mathbf{x}_k^-)^T (M - k)]^T \in \mathbb{R}^7 \quad (19)$$

This is a mild assumption, as deep learning algorithms proved to be able to deal also with partially observable dynamic processes quite effectively [35,41]. The action $\mathbf{a}_k$, instead, corresponds to an instantaneous variation of velocity performed at time t_k

$$\mathbf{a}_k = \Delta \mathbf{v}_k \in [-\Delta v_{\max}, \Delta v_{\max}]^3 \subset \mathbb{R}^3 \quad (20)$$

At the final time $t_f = t_M$, the last $\Delta \mathbf{v}$ is not considered as the output of the closed-loop policy π , but it is computed algebraically in order to match the assigned terminal velocity $\mathbf{v}_f$, unless this would lead to a violation of the constraint in Eq. (5), i.e.,

$$\Delta \mathbf{v}_M = \min\left(\|\mathbf{v}_f - \mathbf{v}_M^-\|, \Delta v_{\max}\right) \frac{\mathbf{v}_f - \mathbf{v}_M^-}{\|\mathbf{v}_f - \mathbf{v}_M^-\|} \quad (21)$$

A. Policy Network

When solving a control problem via deep learning, the control policy π is usually approximated by a DNN with parameters θ , and it is often referred to as π_θ to make the dependence on the network parameters explicit. A DNN is an input-output computing system composed of a (typically large) number of basic mathematical units called *neurons*, arranged into a sequence of *layers*. The DNN adopted in this work is a multilayer perceptron (MLP) network. In MLP networks, each neuron receives as input signals the numerical outputs generated by the units in the previous layer and generates an output signal by applying a (typically nonlinear) *activation function* to a linear combination of all the received inputs. Then, this signal is sent to all the units in the next layer. The whole set of weights and biases used to compute the linear combinations constitute the set of network parameters θ . The input signals received by the neurons of the first (or input) layer are the inputs of the network; the outputs generated by the neurons of the last (or output) layer constitute the network outputs. The intermediate layers are called *hidden layers*.

In the case of a deterministic policy, at each time step t_k , the DNN directly outputs the action $\mathbf{a}_k$, given the observation $\mathbf{o}_k$ of the current system state as input. However, for the present application, a stochastic policy is chosen, as deemed more robust and more oriented toward exploration than a deterministic one. In this case, $\pi_\theta(\mathbf{o}_k)$ should be intended as a shorthand for $\pi_\theta(\mathbf{a}_k | \mathbf{o}_k)$, i.e., the probability of taking action $\mathbf{a}_k$ at time t_k , given the current observation $\mathbf{o}_k$. When a stochastic policy is considered, the DNN returns the parameters of the probability distribution π , and the actual action is sampled from that distribution: $\mathbf{a}_k \sim \pi_\theta(\cdot | \mathbf{o}_k)$. In this work, a diagonal multivariate Gaussian distribution is adopted. The policy network outputs the mean value of the action components $\mu_\theta(\mathbf{o}_k)$, whereas the standard deviations are considered as standalone parameters (i.e., not dependent on the current observation).

Once the network architecture (i.e., number of layers, layer density, and activation functions) is assigned, the problem reduces to the search of the parameters θ that provide an optimal control policy π_θ^* , i.e., a policy that “optimally” solves the considered control problem.

B. Behavioral Cloning Approach

In BC, given a data set $\mathcal{S}$ composed by a large number of reference expert trajectories (i.e., sequences of observation-action pairs), the policy network π_θ^{BC} is trained to imitate, in the best possible way, the expert behavior. This means that, for a given observation, the action taken by the policy should be as close as possible to the one taken by the expert, which is considered as optimal. By leveraging on the DNN interpolation capability, one hopes that the trained policy will be able to achieve high-quality results also on trajectories different (but sufficiently close) from those shown by the expert. In the context of space trajectory optimization, the human expert demonstrations are substituted by open-loop optimal solutions of the control problem provided by a numerical solver. The deterministic OCP in Eq. (13) can be stated as a second-order cone programming problem [38]; thus, it can be solved in a short time using an interior-point algorithm, without requiring any initialization and regardless of the specific problem instance [42].

To make the network able to cope successfully with the considered stochastic perturbations, two different types of expert data sets are investigated. The first data set $\mathcal{S}^F$, called “funnel set,” is constituted

by N_F trajectories τ_j , each one obtained as the solution of the deterministic OCP of Eq. (13) with initial condition randomly sampled according to Eq. (14) in the expected scattering domain. Each expert trajectory $\tau_j \in \mathcal{S}^F$ consists of a sequence of M observation–action pairs $(\mathbf{o}_k, \mathbf{a}_k^*)$, where $\mathbf{a}_k^*$ is the optimal $\Delta \mathbf{v}$ provided by the convex solver when the chaser (observed) state at time t_k is $\mathbf{o}_k$. Solid black lines identify a 20 deg visibility cone, and green squares the nominal initial and final conditions. An example of funnel set, with $N_F = 1000$, is shown in Fig. 3a.

The second data set $\mathcal{S}^E$, called “eyelash set,” is built around the optimal solution $\tau^* = \{(\mathbf{x}_0^*)^*, (\mathbf{x}_1^*)^*, \dots, (\mathbf{x}_M^*)^*\}$ of the original control problem in Eq. (13). More precisely, the eyelash set contains $N_E = LM$ trajectories $\tau_{j,h}$, each one obtained as the solution of the deterministic OCP of Eq. (13), but starting at time t_h with initial condition $(\mathbf{x}_h^-)_j$ scattered around the nominal solution $(\mathbf{x}_h^-)^*$, i.e.,

$$(\mathbf{x}_h^-)_j = (\mathbf{x}_h^-)^* + \delta \mathbf{x} \quad (22)$$

where $\delta \mathbf{x} \sim \mathcal{U}(-\delta \mathbf{x}_i^{\max}, \delta \mathbf{x}_i^{\max})$. Initial conditions that fall outside of the visibility cone are simply ignored and $\delta \mathbf{x}$ resampled until the condition in Eq. (8) is met. So, each expert trajectory $\tau_{j,h} \in \mathcal{S}^E$ consists of $M - h$ optimal observation–action pairs $(\mathbf{o}_k, \mathbf{a}_k^*)$. An example of eyelash set, with $N_E = 3000$, is shown in Fig. 3b.

Once an expert data set $\mathcal{S}$ has been defined, the policy π_{θ}^{BC} is trained to replicate the given behavior by using standard algorithms for supervised learning. The training process works as follows. The trajectories in the considered data set $\mathcal{S}$, of size $|\mathcal{S}| = N$, are randomly split into a training set $\mathcal{T}$ and a validation set $\mathcal{V}$, of size $\eta_t N$ and $(1 - \eta_t)N$, respectively, where the train fraction η_t is a hyperparameter usually set in the range 0.8–0.99. Data in the training set are directly used to train the network, whereas data in the validation set are used to independently assess the network performance (i.e., its capability of predicting the optimal action) on an unseen set of data sampled from the same probability distribution.

To avoid overfitting, the trajectories in the training set are organized in batches of n_b^{BC} trajectories each, and the parameters of the DNN, θ , are optimized by performing, sequentially, a stochastic gradient descent (SGD) on each training batch. The cost function to minimize is the expected mean squared error (MSE), i.e., the squared difference between the policy behavior and the expert behavior

$$J^{\text{BC}}(\theta) = \mathbb{E}_{\tau \in \mathcal{T}} \left[\mathbb{E}_{\mathbf{o}_k \in \tau} \|\mathbf{a}_k^* - \mu_{\theta}^{\text{BC}}(\mathbf{o}_k)\|^2 \right] \quad (23)$$

where the expectation $\mathbb{E}_{\tau \in \mathcal{T}}$ indicates the empirical average over a single batch of expert trajectories, whereas the expectation $\mathbb{E}_{\mathbf{o}_k \in \tau}$ a time average over a single trajectory τ . μ_{θ}^{BC} indicates the mean (i.e., most probable) action of the Gaussian distribution π_{θ}^{BC} . More

precisely, the Adam (adaptive moment estimation) [43] SGD algorithm is used for optimization, with a (constant) learning rate $\alpha^{\text{BC}} = 10^{-4}$. This gradient descent optimization is repeated for n_t training epochs.

In this work, after a preliminary trial-and-error tuning process, the following hyperparameters were used for training the policy network by BC: $n_b^{\text{BC}} = 8$, $\eta_t = 0.95$, $n_t = 1000$. The numbers of expert trajectories in the two data sets are $N_F = 15,000$ and $N_E = 30,000$, which roughly correspond to the same number of observation–action pairs, even though differently distributed over time.

C. Reinforcement Learning Approach

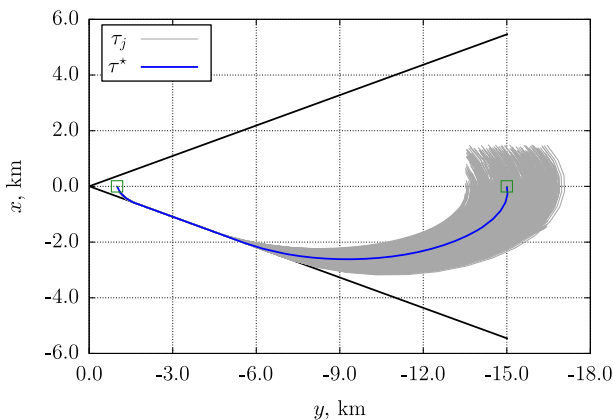
RL involves learning by trial-and-error rather than from expert demonstrations. In RL, a software agent (i.e., the DNN) autonomously learns how to successfully solve a (possibly stochastic) control task, by repeatedly interacting with a (physical or simulated) “environment”, modeled as a discrete-time Markov decision process (MDP), and trying to maximize a utility-based function, or cumulative reward, which plays the role of the merit index of the original OCP. Differently from the BC approach, no precomputed data set of observation–action pairs where to learn from is given to the network; i.e., the agent is not told in advance what actions are optimal in a given set of states. Instead, the agent is left free to explore the environment by itself, attempting admissible actions as a response to either seen (observable MDP) or unseen (partially observable or nonobservable MDP) states, and receiving a scalar reward as immediate feedback. Through interaction with a typically large number of realizations of the environment, the agent collects a wide amount of information, in the form of observation–action–reward tuples, that progressively allows more thoughtful decisions.

In this framework, the final goal of the RL agent is to learn the control policy $(\pi_{\theta}^{\text{RL}})^*$ (i.e., the network parameters θ^*) that maximizes the cumulative return J^{RL} , i.e., the expected value of the (discounted) sum of rewards received along a trajectory

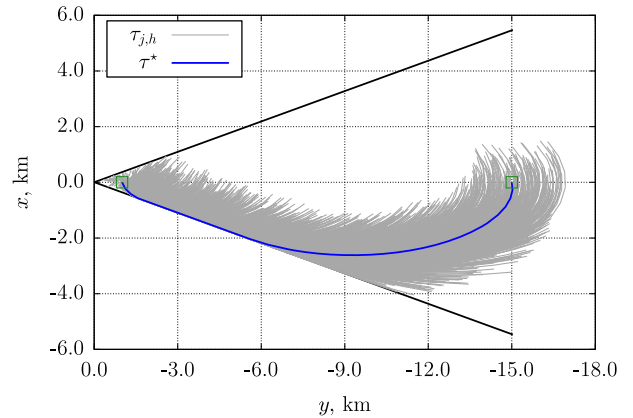
$$J^{\text{RL}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}^{\text{RL}}} \left[\sum_{k=0}^{M-1} \gamma^k R_k \right] \quad (24)$$

where $R_k = R(\mathbf{x}_k^-, \mathbf{a}_k, \mathbf{x}_{k+1}^-)$ is the reward associated with the transition from the state $\mathbf{x}_k^-$ to the next state $\mathbf{x}_{k+1}^-$ due to action $\mathbf{a}_k$, and $\gamma \in (0, 1]$ is a discount factor that is used to either encourage long-term planning ($\gamma = 1$) or short-term rewards ($\gamma \ll 1$). The expectation $\mathbb{E}_{\tau \sim \pi_{\theta}^{\text{RL}}}$ is taken over a batch of trajectories τ , where each trajectory is a sequence of observation–action pairs $\tau = \{(\mathbf{o}_0, \mathbf{a}_0), \dots, (\mathbf{o}_{M-1}, \mathbf{a}_{M-1})\}$ sampled according to the MDP defined by Eqs. (6), (16), and (18–20), with π_{θ}^{RL} as control policy.

Because the goal of the optimization procedure is to minimize the total consumption, while ensuring the compliance with path and



a) Funnel set $\mathcal{S}^F$



b) Eyelash set $\mathcal{S}^E$

Fig. 3 Projections onto the $x - y$ plane of a funnel (a) and eyelash (b) set generated by the convex solver on a sample problem.

terminal constraints, the scalar reward R_k obtained at time t_k was defined as

$$R_k = -\frac{\|\Delta \mathbf{v}_k\|}{M\Delta v_{\max}} - \lambda_1 e_{a,k} - \lambda_2 \max(0, e_{x,k+1} - \varepsilon) - \lambda_3 \max(0, e_{c,k+1} - \varepsilon) \quad (25)$$

where

$$e_{a,k} = \max\{0, \|\Delta \mathbf{v}_k\|/\Delta v_{\max} - 1\} \quad (26)$$

$$e_{x,k} = \begin{cases} 0 & \text{if } k < M \\ \max\{\Delta r/\bar{r}, \Delta v/\bar{v}\} & \text{if } k = M \end{cases} \quad (27)$$

$$e_{c,k} = \max\{0, \max\{\mathbf{C}\mathbf{x}_k^-\}/\bar{r}\} \quad (28)$$

Here $e_{a,k}$ is the (nondimensional) violation at time t_k of constraint in Eq. (5) relative to the maximum Δv magnitude, $e_{c,k}$ is the maximum (nondimensional) violation at time t_k among the four conditions that make up the visibility cone constraint in Eq. (8), and $e_{x,M}$ is the maximum between the final position and velocity error

$$\Delta \mathbf{r} = \|\mathbf{r}_M - \mathbf{r}_f\|, \quad \Delta \mathbf{v} = \|\mathbf{v}_M^+ - \mathbf{v}_f\| \quad (29)$$

made nondimensional with $\bar{r} = 1$ km and $\bar{v} = 0.01$ km/s, respectively; ε represents the tolerance within which the (nondimensional) terminal and visibility cone constraints must be satisfied. More detailed information about the ε value will be provided in Sec. III.D. After a preliminary trial-and-error tuning, the penalty factors $\lambda_1 = 100$ and $\lambda_2 = \lambda_3 = 1$ showed the overall best performance and were selected in the present work.

The RL algorithm adopted in this study is PPO, which is a model-free, policy-gradient, actor-critic method widely recognized for the high performance demonstrated on a number of continuous and high-dimensional control problems [31]. As in all policy-gradient algorithms, the learning process consists of two well-distinguished phases that are repeated iteratively:

1) *Policy rollout*: During this phase the current policy π_{θ}^{RL} is run in n_w independent (e.g., parallel) realizations of the environment (called “workers”) for n_b^{RL} training episodes, each of length M , collecting a batch of $n_w n_b^{\text{RL}}$ trajectories τ .

2) *Policy update*: This is performed by running n_{opt} stochastic gradient ascent (SGA) updates of the DNN parameters θ , by using the on-policy data, i.e., the trajectories collected during the last rollout only, to compute the gradient of the merit index J^{RL} . A slowly decreasing learning rate α^{RL} is generally used to control the step size of the SGA.

The algorithm terminates after a total number of training steps equal to T , i.e., after $N_{\text{upd}} = T/(n_w n_b^{\text{RL}} M)$ policy updates.

In actor-critic methods, two DNNs, named *actor* and *critic*, are used concurrently. Both networks receive as input the current observation $\mathbf{o}_k$, but return different quantities: the actor coincides with the policy network and provides a parameterized control policy π_{θ}^{RL} ; the critic, instead, provides an approximation V_{ϕ} of the value function $V_k^{\text{RL}}(\mathbf{x})$, which is the expected discounted return obtained by starting at time t_k from state $\mathbf{x}$ and acting according to policy π^{RL} until the end of the episode

$$V_k^{\text{RL}}(\mathbf{x}) = \mathbb{E}_{\tau \sim \pi^{\text{RL}}} \left[\sum_{k'=k}^{M-1} \gamma^{k'-k} R_{k'} | \mathbf{x}_k^- = \mathbf{x} \right] \quad (30)$$

Intuitively, the actor controls the agent behavior, whereas the critic evaluates the agent performance and gives a feedback to the actor in order to efficiently update the policy.

In PPO, the actor and critic updates are carried out all at once in the policy update phase by gathering in a unique set of parameters θ the weights and biases of both networks, and by performing SGA on a surrogate objective function J^{PPO} rather than on the merit index J^{RL} . J^{PPO} is defined as

$$J^{\text{PPO}}(\theta) = J^{\text{clip}}(\theta) - c_1 H(\theta) + c_2 S(\theta) \quad (31)$$

with

$$J^{\text{clip}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}^{\text{RL}}} \left[\frac{1}{M} \sum_{k=0}^{M-1} \min\{\tilde{r}_k(\theta) \hat{A}_k, \text{clip}(\tilde{r}_k(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_k\} \right] \quad (32)$$

$$H(\theta) = \frac{1}{2} \mathbb{E}_{\tau \sim \pi_{\theta}^{\text{RL}}} \left[\frac{1}{M} \sum_{k=0}^{M-1} \left(V_{\theta}(\mathbf{o}_k) - \sum_{k'=k}^{M-1} \gamma^{k'-k} R_{k'} \right)^2 \right] \quad (33)$$

$$S(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}^{\text{RL}}} \left[\frac{1}{M} \sum_{k=0}^{M-1} \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}^{\text{RL}}(\cdot | \mathbf{o}_k)} [-\log \pi_{\theta}^{\text{RL}}(\mathbf{a} | \mathbf{o}_k)] \right] \quad (34)$$

The “clipped policy objective function” J^{clip} is used to constrain the updated policy π_{θ}^{RL} to stay in a small range ϵ , named “clip range,” around its previous value $\pi_{\theta_{\text{old}}}^{\text{RL}}$, by clipping the probability that the new policy moves outside of the interval $[(1-\epsilon)\pi_{\theta_{\text{old}}}^{\text{RL}}, (1+\epsilon)\pi_{\theta_{\text{old}}}^{\text{RL}}]$. In this way, the algorithm avoids having too large policy updates during SGA, which are usually the cause of a performance collapse. In particular, in Eq. (32), $\tilde{r}_k(\theta)$ is the probability ratio

$$\tilde{r}_k(\theta) = \frac{\pi_{\theta}^{\text{RL}}(\mathbf{a}_k | \mathbf{o}_k)}{\pi_{\theta_{\text{old}}}^{\text{RL}}(\mathbf{a}_k | \mathbf{o}_k)} \quad (35)$$

while $\hat{A}_k$ denotes the advantage function at time t_k , which is usually approximated by using the so-called generalized advantage estimator (GAE)

$$\hat{A}_k = \sum_{k'=k}^{M-1} (\gamma\lambda)^{k'-k} (R_{k'} + \gamma V_{\theta}(\mathbf{o}_{k'+1}) - V_{\theta}(\mathbf{o}_{k'})) \quad (36)$$

where λ is a hyperparameter named “GAE factor.”

The last two terms in the surrogate objective function J^{PPO} indicate a mean-squared value function error term H , necessary to include the current critic network performance inside the objective function, and an entropy term S , which is usually added to ensure sufficient exploration and prevent premature convergence to a suboptimal deterministic policy [44]. Eventually, c_1 and c_2 are two hyperparameters, named “value function coefficient” and “entropy coefficient,” which control the relative importance of the various terms. The values used for the PPO hyperparameters are reported in Table 1.

D. Implementation Details

The results presented in this work were obtained by using Stable Baselines [45], an open-source library that contains a wide set of improved implementations of RL algorithms based on OpenAI Baselines and gives also the possibility to (pre)train a policy network by BC.

Table 1 PPO hyperparameters

Hyperparameter	Symbol	Value
Training steps	T	96×10^6
Discount factor	γ	0.9999
GAE factor	λ	0.99
Learning rate	α^{RL}	$2.5 \times 10^{-4} (1 - t/T)^a$
Clip range	ϵ	$0.3(1 - t/T)^a$
Value function coefficient	c_1	0.5
Entropy coefficient	c_2	4.75×10^{-8}
Parallel workers	n_w	8
Episodes per batch	n_b^{RL}	4
SGA epochs per update	n_{opt}	30

^aSymbol t indicates the current training step.

The use of fully tested libraries is mandatory in RL, because the results are often significantly impacted by the so-called “code-level optimizations,” i.e., small implementation details seemingly of secondary importance [46]. The use of open-source libraries also allows for an easy reproducibility of the obtained results. In this respect, the whole Python code used to produce the results here presented is made freely available on GitHub.⁸

In this study, the policy network consists of an input layer with $n_i = 7$ neurons, an output layer with $n_o = 3$ neurons, and three hidden layers of size $h_1 = 10n_i$, $h_2 = \sqrt{h_1 h_3}$, and $h_3 = 10n_o$, respectively, as proposed by Gaudet et al. [24]. The activation function is the hyperbolic tangent for all neurons in the hidden layers. For the sake of comparison, the same architecture was used for both the BC and the RL policy networks. The critic network used in RL has the same general architecture, but with $n_o = 1$.

One of the major limitations of RL is its poor constraint handling capability. Indeed, because the MDP formulation only admits a scalar reward function, terminal and path constraints cannot be explicitly accounted for, and all constraint violations must be introduced in the reward function as (weighted) penalty terms [see Eq. (25)]. More specifically, control/state path constraints are often well handled by RL, as the agent receives an immediate (negative) reward whenever it does not meet the imposed constraints. On the contrary, terminal constraints are more challenging, as the delayed (negative) reward given as a response to the constraint violation is the result of all the actions taken during a whole episode (i.e., trajectory). The combined effect of delayed rewards and random initialization of the DNN’s parameters may significantly slow down the convergence, and, consequently, the quality of the attained solution after a fixed number of training steps. A previous study by the authors [41] showed that the use of a penalty-based ε -constraint relaxation method, similar to those sometimes used in stochastic global optimization [47], can greatly improve the success of the RL method. Indeed, in this way, the constraints are gradually enforced during the training process, thus allowing the agent to explore to a greater extent the solution space. So, the constraints satisfaction tolerance ε is decreased during training according to a piecewise constant trend

$$\varepsilon(t) = \begin{cases} 5 \times 10^{-2} & t \leq T/2 \\ 5 \times 10^{-4} & t > T/2 \end{cases} \quad (37)$$

where t is the current training step. The minimum value of the allowed tolerance was selected by looking at similar studies [29]. Indeed, the use of the ε -constraint technique proved to be a simple, yet effective, method to reduce the training sensitivity to the penalty weights λ_2 and λ_3 in the reward function.

IV. Numerical Results

The study case considered in this paper is the Mars Sample Return capture scenario proposed by Hartley [37], where an actively controlled chaser spacecraft has to rendezvous with a target spacecraft that is on a circular Mars orbit of altitude $h \approx 500$ km and angular velocity $\omega = 8.5193 \times 10^{-4}$ rad/s. A V-bar approach is considered: the (nominal) initial state of the chaser is set to $\mathbf{r}_i = [0 \ -15 \ 0]$ km and $\mathbf{v}_i = [0 \ 0 \ 0]$ km/s, and the chaser must stay within a visibility cone with half-angle $\beta = 20^\circ$. The aim of the chaser is to reach, in a time $t_f = 6000$ s, a point slightly away from the target ($\mathbf{r}_f = [0 \ -1 \ 0]$ km) with null relative velocity ($\mathbf{v}_f = [0 \ 0 \ 0]$ km/s). The final capture, which is performed in a free-drift maneuver, is here omitted for the sake of simplicity. The transfer is discretized in $M = 30$ time intervals, of time length $\Delta t = 200$ s each. The maximum thrusting capability of the chaser is $\Delta v_{\max} = 5$ m/s.

The performance of the policies trained by either BC or RL approaches is evaluated by means of closed-loop simulations that account for the presence of scattered initial conditions and stochastic perturbing accelerations. To this end, let us first consider a moderate

level of scattering $\delta \mathbf{x}_i$ on the initial condition, defined as in Eq. (15), with $\delta \mathbf{r}_i^{\max} = [1.5 \ 1.5 \ 0.075]$ km and $\delta \mathbf{v}_i^{\max} = 0$. Furthermore, let $\tilde{\mathbf{a}}_p$ be the unitary perturbation level, corresponding to a random acceleration of $10 \mu\text{m/s}^2$ in the cross-track (x) and in-track (y) directions, and $0.5 \mu\text{m/s}^2$ along the normal direction (z), i.e., $\tilde{\mathbf{a}}_p = [10 \ 10 \ 0.5] \mu\text{m/s}^2$. Six test cases, or environments, named from case A to case F, are thus defined by considering the moderate scattering $\delta \mathbf{x}_i$ on the initial condition plus a perturbing acceleration $\mathbf{a}_p$, sampled from the uniform distribution of Eq. (17), with $\mathbf{a}_p^{\max} = K \tilde{\mathbf{a}}_p$ and $K \in \{0, 1, 5, 10, 20, 50\}$.

All results presented in this section were obtained by using deterministic versions of the policies, i.e., that always take the most probable action $\mu_\theta(\mathbf{o}_k)$ instead of randomly sampling the action from the Gaussian distribution.

A. Training

Six control policies, named $\pi^{\text{RL-A}}, \pi^{\text{RL-B}}, \dots, \pi^{\text{RL-F}}$, are obtained by training an agent in the corresponding environments using the RL approach. Figure 4a shows the learning curves for the considered RL policies, i.e., the plot of the average cumulative reward during the training as a function of the number of training steps. The shaded region around any curve represents the range of variation of the cumulative reward between the n_w parallel workers. Note that the more perturbed the training environment, the lower cumulative reward is attained by the policy. Indeed, the error on the rendezvous conditions is likely to be larger in presence of larger perturbations; consequently, the cumulative reward becomes smaller. As a further remark, all learning curves feature a sudden (small) drop in performance exactly at half the total training time. This is due to the instantaneous reduction in the value of the constraint satisfaction tolerance ε in Eq. (37).

Figure 4b shows the MSE on the training and validation sets as a function of the number of training epochs for the control policies $\pi^{\text{BC-S}^F}$ and $\pi^{\text{BC-S}^E}$ that result from the application of the BC approach to data sets $\mathbf{S}^F$ and $\mathbf{S}^E$, respectively. This plot provides a visual clue that the training process is converging smoothly as both training and validation errors are decreasing. The training is stopped after 1000 epochs, as the reduction in the MSE becomes negligible beyond this point.

The network training by RL took about 7 h with eight parallel workers on a computer equipped with an 8-core Intel Core i7-9700K CPU @3.60 GHz, whereas the training by BC about 45 minutes on the same hardware. It is here worthwhile to remark that, once the training is complete, the computational cost for evaluating the control in the simulations is negligible, being equivalent to that of a few small-size matrix multiplications. Hence, the time needed for generating a single optimal trajectory is essentially driven by the solution of the (linear) equations of motions.

B. Nominal Trajectories

The nominal trajectories, obtained by running the RL and BC policies in the deterministic unperturbed scenario, are presented in Fig. 5a. The optimal problem solution provided by the convex approach is also reported for the sake of comparison. Differences with respect to the convex solution are minimal, and all the trajectories look almost identical. Figure 5b shows the distribution of the Δv s over the transfer for the different policies. A bang-bang structure is apparent in the convex solution. A singular arc appears where the path constraint enforcing the visibility cone [Eq. (8)] is active over a finite time interval ($t_k \in [3400, 5200]$ s). This structure is only partially recovered by the RL and BC solutions. Indeed, this suboptimal distribution of the thrust over the flight time is responsible for their overall higher consumption.

Table 2 presents, for each nominal trajectory (and for the optimal convex solution, named “CVX”), the total consumption, ΔV , the position error at the rendezvous, Δr , and the maximum angular distance of the trajectory from the visibility cone, $\Delta\beta = \beta_{\max} - \tilde{\beta}$, where $\beta_{\max}$ is the half-angle of the smallest visibility cone that would contain the whole chaser trajectory

⁸<https://github.com/LorenzoFederici/DL-rendezvous-guidance>.

$$\beta_{\max} = \max_{k \in [0, M]} \arctan(\max\{|x_k/y_k|, |z_k/y_k|\}) \quad (38)$$

The error on the final velocity is always null for all policies, and it is thus omitted both here and in what follows. This is because the last $\Delta \mathbf{v}$ is computed algebraically as a difference between the final chaser velocity and the assigned terminal velocity [see Eq. (21)]. Thus, the terminal velocity constraint is automatically satisfied once the computed $\Delta \mathbf{v}$ has a magnitude lower than the maximum admitted by constraint in Eq. (5). This condition happened to be always true, as the $\Delta v_{\max}$ is significantly larger (at least three times) than the maximum single $\Delta \mathbf{v}$ performed by the chaser in the considered mission.

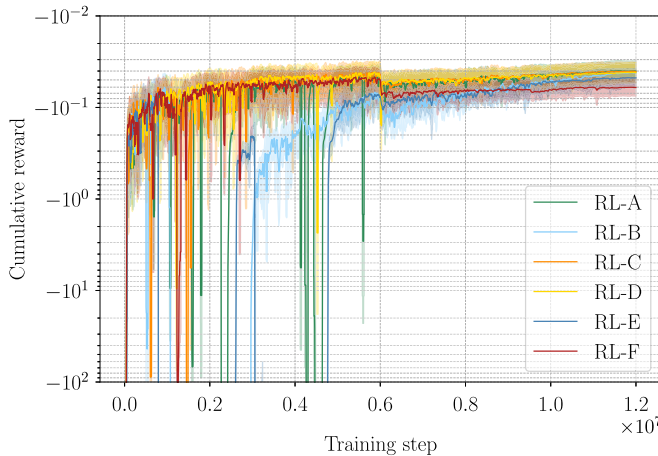
The results in Table 2 highlight that all RL solutions are more expensive than those provided by the BC policies. On the other hand, the terminal position error is comparable for most policies, but for the policy $\text{BC-}\mathcal{S}^F$, which features a position error that is larger than $\text{BC-}\mathcal{S}^E$'s by two orders of magnitude. Indeed, despite that the two training sets are made up of roughly the same number of observation–action pairs, in set $\mathcal{S}^F$ these samples are uniformly distributed over time (i.e., the same number of pairs is referred to the M time steps), whereas in set $\mathcal{S}^E$ their number linearly increases with time (the number of pairs at time step t_h is Mh). This characteristic allows policy $\text{BC-}\mathcal{S}^E$ to become more accurate while approaching the rendezvous point. Moreover, Fig. 3a shows that all the expert trajectories in the data set $\mathcal{S}^F$ are “piled” on the cone constraint when close to the final time, thus providing a poor information content in the surrounding space. This is not the case of data set $\mathcal{S}^E$, which offers a better coverage of the observation–action space.

The terminal position error is higher for policy RL-F too. Indeed, this value is comparable to the error directly due to the presence of the

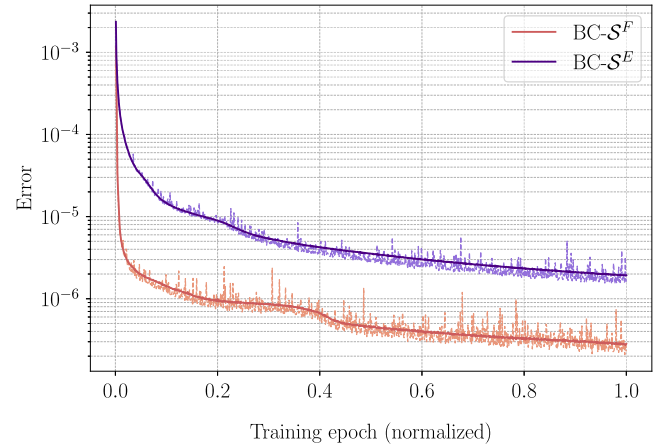
Table 2 Overview of the nominal trajectories

Policy	ΔV , m/s	Δr , m	$\Delta \beta$, deg
$\text{BC-}\mathcal{S}^F$	4.884	16.175	0.250
$\text{BC-}\mathcal{S}^E$	4.994	0.367	0.050
RL-A	5.230	0.329	−0.143
RL-B	5.431	0.155	−0.171
RL-C	5.225	0.269	−0.153
RL-D	5.226	0.384	−0.148
RL-E	5.348	0.431	−0.373
RL-F	5.631	2.841	−0.513
CVX	4.877	0.000	0.000

sole perturbing acceleration in the last trajectory segment. This unintended behavior is probably related to the adopted penalty weights in the reward function [Eq. (25)]: the agent learned that, in presence of a high perturbing acceleration, a Δv_M slightly lower than the correct one (as confirmed by Fig. 5b) is able to provide the best compromise between the error on the final position and the total ΔV consumption. The importance of properly tuning such penalty weights is thus apparent. Note that the RL-F solution also features the largest total ΔV . Indeed, the trajectory is well within the visibility cone ($\Delta \beta = -0.513$ deg); hence it does not exploit the constraint to the maximum in order to be able to cope with potentially large perturbing accelerations. The use of the same penalty weights in the reward function regardless of the magnitude of the perturbing acceleration leads to an unexpected condition, where the policies RL-C and RL-D

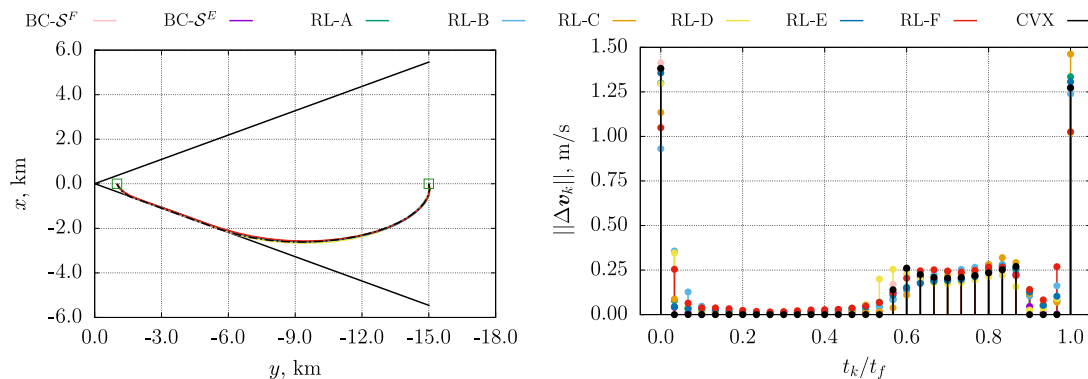


a) Learning curves of RL policies. Values smoothed with a moving average on a 1000-step window



b) Training error (solid line) and validation error (dashed line) obtained with the two BC policies

Fig. 4 Convergence behavior of RL policies (a) and BC policies (b) during training.



a) Nominal trajectories corresponding to different policies **b) Distribution of the $\Delta \mathbf{v}$ s along the nominal trajectories**

Fig. 5 Performance of the policies in the deterministic environment.

show a slightly lower total ΔV than the policy RL-A, which was trained in the unperturbed environment (case A). Besides possible performance fluctuations related to the intrinsic randomness of the RL method, this may indicate that the tuning is slightly more favorable to RL-C.

C. Monte Carlo Analysis

Eventually, the performance and robustness of all policies are evaluated by means of a Monte Carlo analysis involving 1000 closed-loop simulations, over the six test cases previously defined, which feature perturbing accelerations of increasing magnitude. A model predictive controller that runs the optimal convex policy, i.e., solves the second-order cone programming problem at every step, is used as a benchmark. Several figure of merits are recorded, namely, the average total consumption ΔV_{avg} of the Monte Carlo samples, and the $\Delta V_{1\sigma}$, $\Delta V_{2\sigma}$, and $\Delta V_{3\sigma}$, which correspond to the minimum value of the ΔV for which the 68.3, 95.5, and 99.7% of the solutions fall below that threshold, respectively. The same thresholds are used to define the figures of merit $\Delta r_{1\sigma}$, $\Delta r_{2\sigma}$, and $\Delta r_{3\sigma}$ related to the error on the rendezvous position. Finally, the maximum angular distance $(\Delta\beta)_{\text{max}}$ from the visibility cone among the different Monte Carlo trajectories is also recorded.

Tables 3–5 report the values of these figures of merit for the test cases A, D, and F, which correspond to the case of none, medium, and

highest perturbing acceleration, respectively. The policy BC- $\mathcal{S}^F$ is omitted as it performs significantly worse than all other policies, featuring $\Delta r_{3\sigma} > 100$ m and $(\Delta\beta)_{\text{max}} > 3$ deg in all test cases. Indeed, Fig. 6 presents the trajectories resulting from the Monte Carlo analysis in test case D (medium perturbing acceleration) for the two BC policies. It is apparent that the policy BC- $\mathcal{S}^F$ violates the visibility cone constraint and leads to some errors on the arrival point. Instead, the policy BC- $\mathcal{S}^E$ meets the requirements with greater precision.

Figure 7 provides a clear visualization of the same results by means of a scatter plot matrix. Only the values $\Delta V_{3\sigma}$, $\Delta r_{3\sigma}$, and $(\Delta\beta)_{\text{max}}$ are reported for the sake of conciseness. Each curve represents a different policy, with the dots indicating the test cases (from A to F moving along the curve from the bottom to the top).

The results of the Monte Carlo analysis indicate that the policy BC- $\mathcal{S}^E$ is the least expensive one; indeed, it features the minimum ΔV_{avg} and $\Delta V\sigma$ -contours, which are close to those of the MPC solution, in all test cases but case F, where its performance starts to worsen in terms of ΔV . On the other hand, the error on the final position in case F is the slightest one. This behavior derives from the fact that BC- $\mathcal{S}^E$ was trained by using propellant-optimal trajectories, which always meet the terminal position constraint with great accuracy.

Policies RL-A and RL-B share the title of the most expensive policy, showing always the largest $\Delta V_{2\sigma}$ and $\Delta V_{3\sigma}$, despite that their average ΔV is quite good. These policies are trained in environments with no or small presence of perturbations. Thus, when subjected to greater disturbances, outliers tend to show up, affecting significantly the external σ contours.

Policy RL-F performs well only in case F, i.e., in the environment it was trained on. It safely respects the visibility cone constraint; for this reason, it requires a slightly larger ΔV than other policies, as already outlined for the nominal deterministic trajectory.

This extended Monte Carlo analysis showed that all policies, but BC- $\mathcal{S}^F$, provide similar closed-loop guidance performance as measured by ΔV consumption and constraint satisfaction. The worse performance of BC- $\mathcal{S}^F$ is related to the poor coverage of the observation–action space provided by data set $\mathcal{S}^F$. Indeed, as already outlined, the expert trajectories tend to pile up on the cone constraint near the rendezvous point, making the network unable to correctly control the chaser when, because of the perturbing acceleration, it starts moving away from the cone edge near the singular-arc region. Conversely, BC- $\mathcal{S}^E$, which is trained on a data set that covers more uniformly the observation–action space, often provides the best performance in terms of both ΔV and terminal position error Δr . The most evident drawback of BC- $\mathcal{S}^E$ concerns the inability to satisfy the visibility cone constraint in all test cases. Indeed, being trained on the deterministic solution, it keeps pushing the trajectory toward the edge of the cone, and often exceeding it, yet slightly, in presence of perturbations. On the other hand, RL-based policies are more robust, as they were trained on the perturbed environment directly. For this reason, the visibility cone constraint is met with a greater margin. Unfortunately, RL suffers from the need of carefully selecting a few hyperparameters: first and foremost, the learning rate and the penalty weights in the reward function, which call for a longer, and often nontrivial, trial-and-error tuning process. From a computational point of view, the network training by RL is also much slower (about seven times) than by BC. This aspect is apparent in the considered problem, where the expert trajectories were generated almost at no cost by solving the OCP using a convex programming algorithm. Further investigations are required to understand the proper tradeoff between the number of expert trajectories and the BC performance, in case of nonconvex problems, where the computational effort associated to the OCP solution is significant. A possible mix between the BC and RL approaches, attained by pretraining the RL agent with a minimum number of precomputed optimal solutions, also deserves to be investigated.

A final comment concerns the viability of deep learning techniques for dealing with problem of increasing complexity, which is a major point in motivating the research on these techniques. Being a model-free algorithm, RL is expected to transition seamlessly from the

Table 3 Results of the closed-loop Monte Carlo analysis: test case A

Policy	ΔV_{avg} , m/s	$\Delta V_{1\sigma}$, m/s	$\Delta V_{2\sigma}$, m/s	$\Delta V_{3\sigma}$, m/s	$\Delta r_{1\sigma}$, m	$\Delta r_{2\sigma}$, m	$\Delta r_{3\sigma}$, m	$(\Delta\beta)_{\text{max}}$, deg
BC- $\mathcal{S}^E$	5.426	5.916	7.552	8.253	0.423	0.493	0.656	0.066
RL-A	5.733	6.171	8.160	9.158	0.391	0.630	0.731	−0.120
RL-B	5.872	6.321	8.326	9.159	0.250	0.490	0.872	−0.101
RL-C	5.715	6.066	7.859	8.568	0.333	0.384	0.501	−0.077
RL-D	5.673	6.101	7.876	8.695	0.487	0.756	1.191	−0.095
RL-E	5.925	6.225	8.173	8.850	0.421	0.469	0.476	−0.292
RL-F	6.088	6.477	8.243	8.931	3.367	4.414	5.365	−0.404
MPC	5.283	5.744	7.424	8.049	0.000	0.000	0.000	0.000

Table 4 Results of the closed-loop Monte Carlo analysis: test case D

Policy	ΔV_{avg} , m/s	$\Delta V_{1\sigma}$, m/s	$\Delta V_{2\sigma}$, m/s	$\Delta V_{3\sigma}$, m/s	$\Delta r_{1\sigma}$, m	$\Delta r_{2\sigma}$, m	$\Delta r_{3\sigma}$, m	$(\Delta\beta)_{\text{max}}$, deg
BC- $\mathcal{S}^E$	5.584	6.017	7.703	8.426	3.734	4.873	5.662	0.230
RL-A	5.770	6.187	8.174	9.190	3.778	4.949	5.851	0.283
RL-B	5.901	6.317	8.321	9.189	3.752	4.918	5.597	0.201
RL-C	5.754	6.083	7.906	8.617	3.736	4.912	5.690	0.103
RL-D	5.720	6.123	7.925	8.731	3.724	5.005	5.824	−0.003
RL-E	5.952	6.236	8.159	8.884	3.743	4.862	5.522	−0.155
RL-F	6.106	6.478	8.243	8.964	5.264	7.297	8.828	−0.313
MPC	5.538	6.019	7.655	8.275	3.706	4.846	5.479	0.149

Table 5 Results of the closed-loop Monte Carlo analysis: test case F

Policy	ΔV_{avg} , m/s	$\Delta V_{1\sigma}$, m/s	$\Delta V_{2\sigma}$, m/s	$\Delta V_{3\sigma}$, m/s	$\Delta r_{1\sigma}$, m	$\Delta r_{2\sigma}$, m	$\Delta r_{3\sigma}$, m	$(\Delta\beta)_{\text{max}}$, deg
BC- $\mathcal{S}^E$	6.667	6.960	8.695	9.592	18.676	24.429	27.659	0.910
RL-A	6.309	6.649	8.653	10.043	19.241	25.857	32.790	2.182
RL-B	6.387	6.795	8.726	10.363	19.228	26.180	35.262	2.497
RL-C	6.419	6.766	8.599	9.792	19.307	26.393	34.392	0.997
RL-D	6.387	6.727	8.589	9.685	18.942	25.074	28.568	0.916
RL-E	6.442	6.720	8.630	9.764	18.653	24.866	28.105	0.696
RL-F	6.499	6.872	8.628	9.755	19.104	25.023	28.827	0.081
MPC	6.748	7.163	8.637	9.562	18.570	24.265	27.642	0.808

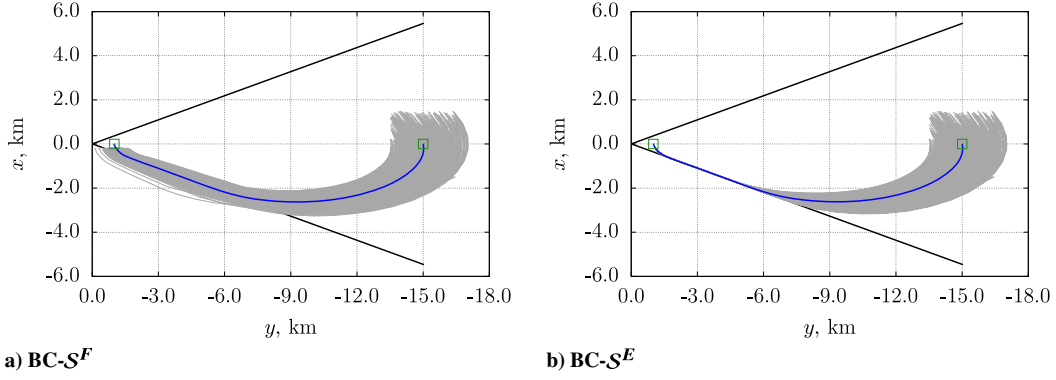


Fig. 6 Monte Carlo simulations (in gray) realized by running the two BC policies in 1000 realizations of environment D. The nominal deterministic trajectory is reported in blue.

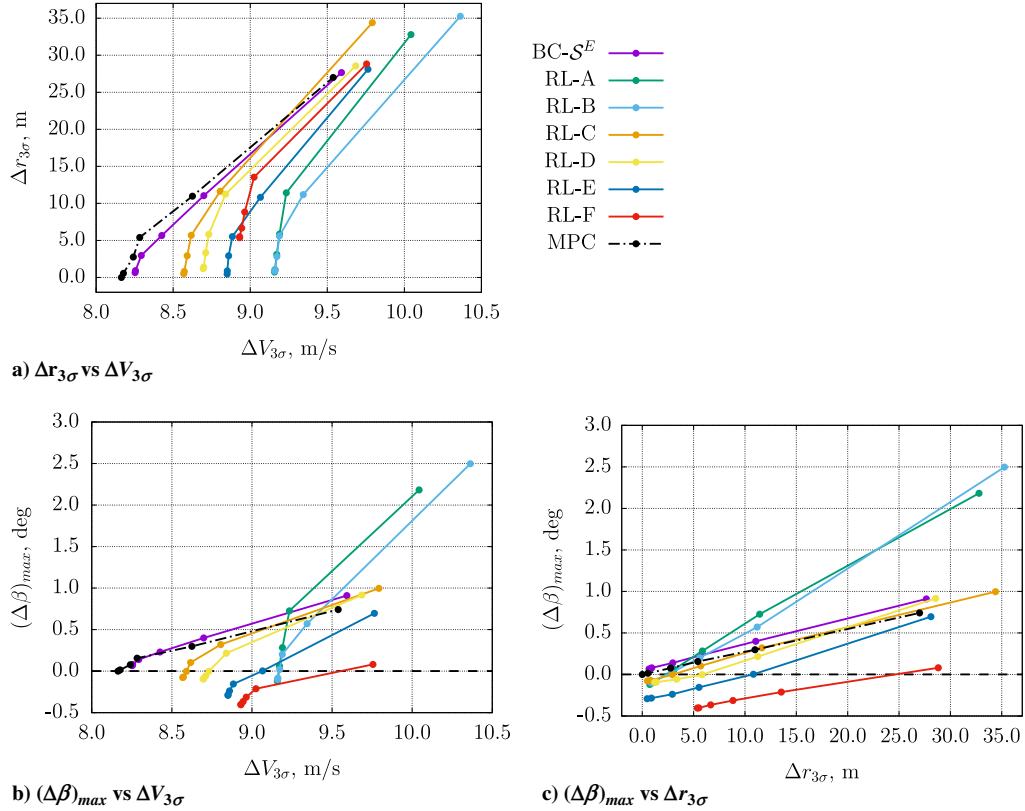


Fig. 7 Results of the closed-loop Monte Carlo analysis. Scatter plot matrix.

linearized dynamics, here used to describe the relative motion of the chaser with respect to the target, to higher fidelity gravitational models and/or to mission scenarios that account for additional constraints on the allowed thrust direction. The same property does not hold for traditional, MPC-based methods. These methods are capable of achieving highly accurate optimal solutions in real time on simplified problems, but may start losing effectiveness as the problem grows in complexity. In fact, because an OCP must be solved online at each control cycle, a reduction of the control update frequency is expected when the problem to be solved becomes more complex. BC places somehow in between RL- and MPC-based methods, as it relies on a set of expert demonstrations that must be provided externally to the network. Thus, if the number of expert demonstrations is sufficiently large and representative of the solution space, the quality of the control law is close to the one obtained by MPC-based method, as the reported results clearly show, but with the additional advantage it can run at a high-frequency rate in real-time, regardless of the problem complexity. In fact, the computational burden is almost entirely concentrated in the generation of the expert trajectories and

network training phases, both of which are performed on the ground before the start of the mission, and can be thus carried out in parallel on high-performance workstations. The cost for evaluating the control using a trained network (by either BC or RL) is substantially negligible regardless of the problem complexity and does not impact the control frequency even on low-cost flight hardware.

V. Conclusions

In this paper, the performance and robustness of guidance algorithms based on deep learning techniques were investigated. A well-known problem in spaceflight mechanics, i.e., the linear multi-impulsive rendezvous, served as a basis for comparison of several learning approaches. Because the solution to the considered problem instance can be readily retrieved via convex programming algorithms, the achieved results were not only compared with each other, but also with the optimal solution, thus providing a comprehensive overview of the effectiveness and validity of the studied techniques.

This paper examined two methods: BC and RL. Both were shown to be viable tools for the autonomous guidance of a chaser spacecraft that is reaching for a passive target. Indeed, these tools can provide a control policy that meets the mission requirements within acceptable tolerances (although with room for improvements) while keeping the propellant consumption to a minimum. The major appeal of the investigated approaches is the in-flight computational cost close to zero. In fact, all the computational effort is associated with the training of the neural network, which is performed offline (i.e., preflight); thus, it can be done on high-performance computing hardware.

BC was shown to be able to train high-performance policies, provided that a sufficiently heterogeneous set of expert trajectories is adopted for the training. Indeed, feeding the network with observation–action pairs too similar to each other may cause overfitting, and makes the network unable to generalize well. For instance, the network fails to predict the optimal action even for observations that slightly deviate from the expert demonstrations used for training. RL, on the other hand, does not suffer from such an issue, as it does not require any expert information to set up the training. This is quite an appealing property when dealing with problems for which generating a set of optimal solutions may be too cumbersome or expensive, but it also significantly increases the computational burden of training as the exploration of the environment usually requires a larger number of training steps.

The robustness of the devised policies with respect to off-nominal initial conditions and random in-flight disturbances was investigated via an extensive Monte Carlo analysis. Simulations showed that, after a proper training, most policies can autonomously compensate for the effect of the perturbations, thus enabling the chaser to reach a desired terminal state with an acceptable propellant cost even in nondeterministic scenarios. Even though the problem at hand can be deemed as particularly simple, the same methodology can be applied to more complex problems with only minor implementation changes. Indeed, neural networks proved to be able to scale well on problems of arbitrarily large complexity and size. Combining the two considered methods may also lead to a significant reduction of the computational burden, as BC can provide, in a relatively brief time, quite an accurate starting point to initialize the RL process.

In conclusion, the obtained results are highly encouraging, as robust optimal control policies were derived for the rendezvous problem using either BC or RL. Because reliability and efficiency are key features of any guidance scheme, these machine learning techniques hold the promise to play a pivotal role in the design of future aerospace applications with unprecedented levels of autonomy and robustness.

References

- [1] Woffinden, D. C., and Geller, D. K., "Navigating the Road to Autonomous Orbital Rendezvous," *Journal of Spacecraft and Rockets*, Vol. 44, No. 4, 2007, pp. 898–909.
<https://doi.org/10.2514/1.30734>
- [2] Lu, P., "Introducing Computational Guidance and Control," *Journal of Guidance, Control, and Dynamics*, Vol. 40, No. 2, 2017, pp. 193–193.
<https://doi.org/10.2514/1.G002745>
- [3] Bodin, P., Noteborn, R., Larsson, R., and Chasset, C., "System Test Results from the GNC Experiments on the PRISMA In-Orbit Test Bed," *Acta Astronautica*, Vol. 68, No. 7, 2011, pp. 862–872.
<https://doi.org/10.1016/j.actaastro.2010.08.021>
- [4] Pierson, H. A., and Gashler, M. S., "Deep Learning in Robotics: A Review of Recent Research," *Advanced Robotics*, Vol. 31, No. 16, 2017, pp. 821–835.
<https://doi.org/10.1080/01691864.2017.1365009>
- [5] Sallab, A. E., Abdou, M., Perot, E., and Yogamani, S., "Deep Reinforcement Learning Framework for Autonomous Driving," *Electronic Imaging*, Vol. 2017, No. 19, 2017, pp. 70–76.
<https://doi.org/10.2352/ISSN.2470-1173.2017.19.AVM-023>
- [6] Yan, C., Xiang, X., and Wang, C., "Towards Real-Time Path Planning Through Deep Reinforcement Learning for a UAV in Dynamic Environments," *Journal of Intelligent & Robotic Systems*, Vol. 98, No. 2, 2020, pp. 297–309.
<https://doi.org/10.1007/s10846-019-01073-3>
- [7] Choi, U., and Ahn, J., "Imitation Learning-Based Unmanned Aerial Vehicle Planning for Multitarget Reconnaissance Under Uncertainty," *Journal of Aerospace Information Systems*, Vol. 17, No. 1, 2020, pp. 36–50.
<https://doi.org/10.2514/1.1010759>
- [8] Loquercio, A., Kaufmann, E., Ranftl, R., Dosovitskiy, A., Koltun, V., and Scaramuzza, D., "Deep Drone Racing: From Simulation to Reality with Domain Randomization," *IEEE Transactions on Robotics*, Vol. 36, No. 1, 2020, pp. 1–14.
<https://doi.org/10.1109/TRO.2019.2942989>
- [9] Izzo, D., Märtens, M., and Pan, B., "A Survey on Artificial Intelligence Trends in Spacecraft Guidance Dynamics and Control," *Astrodynamics*, Vol. 3, No. 4, 2019, pp. 287–299.
<https://doi.org/10.1007/s42064-018-0053-6>
- [10] Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., and Ghemawat, S., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015, <https://www.tensorflow.org/>.
- [11] Izzo, D., Öztürk, E., and Märtens, M., "Interplanetary Transfers via Deep Representations of the Optimal Policy and/or of the Value Function," *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, Association for Computing Machinery, New York, 2019, pp. 1971–1979.
<https://doi.org/10.1145/3319619.3326834>
- [12] Sánchez-Sánchez, C., Izzo, D., and Hennes, D., "Learning the Optimal State-Feedback Using Deep Networks," *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, Inst. of Electrical and Electronics Engineers, New York, 2016, pp. 1–8.
<https://doi.org/10.1109/ssci.2016.7850105>
- [13] Sánchez-Sánchez, C., and Izzo, D., "Real-Time Optimal Control via Deep Neural Networks: Study on Landing Problems," *Journal of Guidance, Control, and Dynamics*, Vol. 41, No. 5, 2018, pp. 1122–1135.
<https://doi.org/10.2514/1.G002357>
- [14] Cheng, L., Wang, Z., Jiang, F., and Li, J., "Fast Generation of Optimal Asteroid Landing Trajectories Using Deep Neural Networks," *IEEE Transactions on Aerospace and Electronic Systems*, Vol. 56, No. 4, 2020, pp. 2642–2655.
<https://doi.org/10.1109/TAES.2019.2952700>
- [15] Cheng, L., Wang, Z., Song, Y., and Jiang, F., "Real-Time Optimal Control for Irregular Asteroid Landings Using Deep Neural Networks," *Acta Astronautica*, Vol. 170, May 2020, pp. 66–79.
<https://doi.org/10.1016/j.actaastro.2019.11.039>
- [16] Shi, Y., and Wang, Z., "A Deep Learning-Based Approach to Real-Time Trajectory Optimization for Hypersonic Vehicles," *AIAA Scitech 2020 Forum*, AIAA Paper 2020-0023, 2020.
<https://doi.org/10.2514/6.2020-0023>
- [17] Shi, Y., and Wang, Z., "Onboard Generation of Optimal Trajectories for Hypersonic Vehicles Using Deep Learning," *Journal of Spacecraft and Rockets*, Vol. 58, No. 2, 2021, pp. 400–414.
<https://doi.org/10.2514/1.A34670>
- [18] Cheng, L., Wang, Z., Jiang, F., and Zhou, C., "Real-Time Optimal Control for Spacecraft Orbit Transfer via Multiscale Deep Neural Networks," *IEEE Transactions on Aerospace and Electronic Systems*, Vol. 55, No. 5, 2019, pp. 2436–2450.
<https://doi.org/10.1109/TAES.2018.2889571>
- [19] Izzo, D., and Öztürk, E., "Real-Time Guidance for Low-Thrust Transfers Using Deep Neural Networks," *Journal of Guidance, Control, and Dynamics*, Vol. 44, No. 2, 2021, pp. 315–327.
<https://doi.org/10.2514/1.G005254>
- [20] Ross, S., Gordon, G., and Bagnell, D., "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.
- [21] Furfaro, R., Bloise, I., Orlandelli, M., Di Lizia, P., Toppo, F., and Linares, R., "Deep Learning for Autonomous Lunar Landing," *Advances in the Astronautical Sciences*, Vol. 167, 2018, pp. 3285–3306.
- [22] Miller, D., Englander, J. A., and Linares, R., "Interplanetary Low-Thrust Design Using Proximal Policy Optimization," *Advances in the Astronautical Sciences*, Vol. 171, 2020, pp. 1575–1592.
- [23] Holt, H., Armellin, R., Scorsoglio, A., and Furfaro, R., "Low-Thrust Trajectory Design Using Closed-Loop Feedback-Driven Control Laws and State-Dependent Parameters," *AIAA Scitech 2020 Forum*, AIAA Paper 2020-1694, 2020.
<https://doi.org/10.2514/6.2020-1694>
- [24] Gaudet, B., Linares, R., and Furfaro, R., "Deep Reinforcement Learning for Six Degree-of-Freedom Planetary Landing," *Advances in Space Research*, Vol. 65, No. 7, 2020, pp. 1723–1741.
<https://doi.org/10.1016/j.asr.2019.12.030>

- [25] Scorsoglio, A., Furfaro, R., Linares, R., and Massari, M., "Actor-Critic Reinforcement Learning Approach to Relative Motion Guidance in Near-Rectilinear Orbit," *Advances in the Astronautical Sciences*, Vol. 168, 2019, pp. 1737–1756.
- [26] Sullivan, C. J., and Bosanac, N., "Using Reinforcement Learning to Design a Low-Thrust Approach into a Periodic Orbit in a Multi-Body System," *AIAA Scitech 2020 Forum*, AIAA Paper 2020-1914, 2020. <https://doi.org/10.2514/6.2020-1914>
- [27] LaFarge, N. B., Miller, D., Howell, K. C., and Linares, R., "Guidance for Closed-Loop Transfers Using Reinforcement Learning with Application to Libration Point Orbits," *AIAA Scitech 2020 Forum*, AIAA Paper 2020-0458, 2020. <https://doi.org/10.2514/6.2020-0458>
- [28] Gaudet, B., Furfaro, R., and Linares, R., "Reinforcement Learning for Angle-Only Intercept Guidance of Maneuvering Targets," *Aerospace Science and Technology*, Vol. 99, April 2020, Paper 105746. <https://doi.org/10.1016/j.ast.2020.105746>
- [29] Broida, J., and Linares, R., "Spacecraft Rendezvous Guidance in Cluttered Environments via Reinforcement Learning," *Advances in the Astronautical Sciences*, Vol. 168, April 2019, pp. 1777–1788.
- [30] Hovell, K., and Ulrich, S., "On Deep Reinforcement Learning for Spacecraft Guidance," *AIAA Scitech 2020 Forum*, AIAA Paper 2020-1600, 2020. <https://doi.org/10.2514/6.2020-1600>
- [31] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017.
- [32] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D., "Continuous Control with Deep Reinforcement Learning," arXiv preprint arXiv:1509.02971, 2015.
- [33] Gaudet, B., Linares, R., and Furfaro, R., "Six Degree-of-Freedom Body-Fixed Hovering over Unmapped Asteroids via LIDAR Altimetry and Reinforcement Meta-Learning," *Acta Astronautica*, Vol. 172, July 2020, pp. 90–99. <https://doi.org/10.1016/j.actaastro.2020.03.026>
- [34] Gaudet, B., Linares, R., and Furfaro, R., "Terminal Adaptive Guidance via Reinforcement Meta-Learning: Applications to Autonomous Asteroid Close-Proximity Operations," *Acta Astronautica*, Vol. 171, June 2020, pp. 1–13. <https://doi.org/10.1016/j.actaastro.2020.02.036>
- [35] Scorsoglio, A., Furfaro, R., Linares, R., and Gaudet, B., "Image-Based Deep Reinforcement Learning for Autonomous Lunar Landing," *AIAA Scitech 2020 Forum*, AIAA Paper 2020-1910, 2020. <https://doi.org/10.2514/6.2020-1910>
- [36] Clohessy, W., and Wiltshire, R. S., "Terminal Guidance System for Satellite Rendezvous," *Journal of the Aerospace Sciences*, Vol. 27, No. 9, 1960, pp. 653–658. <https://doi.org/10.2514/8.8704>
- [37] Hartley, E. N., "A Tutorial on Model Predictive Control for Spacecraft Rendezvous," *2015 European Control Conference (ECC)*, IEEE, New York, 2015, pp. 1355–1361. <https://doi.org/10.1109/ECC.2015.7330727>
- [38] Benedikter, B., and Zavoli, A., "Convex Optimization of Linear Impulsive Rendezvous," arXiv preprint 1912.08038, 2019.
- [39] *Gurobi Optimizer Reference Manual*, Gurobi Optimization, LLC, 2020.
- [40] Richards, A., and How, J., "Performance Evaluation of Rendezvous Using Model Predictive Control," *AIAA Guidance, Navigation, and Control Conference and Exhibit*, AIAA Paper 2003-5507, 2003. <https://doi.org/10.2514/6.2003-5507>
- [41] Zavoli, A., and Federici, L., "Reinforcement Learning for Low-Thrust Trajectory Design of Interplanetary Missions," *AAS/AIAA Astrodynamics Specialist Conference*, AAS Paper 20-616, 2020.
- [42] Boyd, S., Boyd, S. P., and Vandenberghe, L., *Convex Optimization*, Cambridge Univ. Press, Cambridge, U.K., 2004, pp. 1–630. <https://doi.org/10.1017/CBO9780511804441>
- [43] Kingma, D. P., and Ba, J., "Adam: A Method for Stochastic Optimization," arXiv preprint arXiv:1412.6980, 2014.
- [44] Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K., "Asynchronous Methods for Deep Reinforcement Learning," *International Conference on Machine Learning*, 2016, pp. 1928–1937.
- [45] Hill, A., Raffin, A., Ernestus, M., Gleave, A., Kanervisto, A., Traore, R., Dhariwal, P., Hesse, C., Klimov, O., Nichol, A., Plappert, M., Radford, A., Schulman, J., Sidor, S., and Wu, Y., "Stable Baselines," 2018, <https://github.com/hill-a/stable-baselines>.
- [46] Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., and Madry, A., "Implementation Matters in Deep RL: A Case Study on PPO and TRPO," *International Conference on Learning Representations*, 2020.
- [47] Federici, L., Benedikter, B., and Zavoli, A., "EOS: A Parallel, Self-Adaptive, Multi-Population Evolutionary Algorithm for Constrained Global Optimization," *2020 IEEE Congress on Evolutionary Computation (CEC)*, IEEE, New York, 2020, pp. 1–10. <https://doi.org/10.1109/CEC48606.2020.9185800>

C. W. Roscoe
Associate Editor